

SMART WHEELCHAIR: AUTONOMOUS PATIENT NAVIGATION IN HOSPITAL ENVIRONMENTS

Semester Project Report — Georgios Angelidakis 2022030062
Academic Year 2025 – 2026, Autonomous Agents

1. Theme / Problem

1.1 Context

Hospital environments present significant challenges in urgent patient transportation, for accommodated patients. The manual operation of wheelchairs by staff members consumes valuable time and human resources, particularly in large hospital complexes where distances between departments can be considerable. Delays in patient transport — especially in time-critical scenarios such as emergency room admissions — can have direct consequences on patient outcomes and overall care quality.

1.2 Problem Statement

This project addresses the following core problem:

Core Problem

How can an autonomous wheelchair agent navigate a hospital environment intelligently, responding dynamically to patient transport requests across multiple departments, while avoiding obstacles and adapting to unpredictable layouts?

Specifically, the project focuses on three interconnected challenges:

- Efficient pathfinding in a complex, obstacle-rich indoor environment
- Autonomous decision-making for routing based on simulated patient needs
- Real-time visualization and user interaction within a 3D simulation environment

2. Approach / Solution

2.1 Technology Stack

The simulation was developed entirely within the Unity 3D engine, leveraging its robust physics system, scene management tools, and UI framework. The project is implemented in C# programming language. Furthermore, all the exogenous assets included in the scene are accompanied by **Creative Commons 0 license**.

Technology	Purpose
Unity 3D / Unity Physics	Core simulation engine and 3D environment rendering. Collision simulation between objects.
C# Scripts	All pathfinding, agent, and UI logic
Unity NavMesh (grid-based)	Custom grid overlay for flow field generation
3D Assets / Textures	Realistic hospital scene

2.2 Flow Field Pathfinding

The core navigation system is built on a Flow Field pathfinding algorithm. Unlike traditional A* or Dijkstra pathfinding which computes a path for a single agent, flow fields generate a vector direction for every cell in the grid simultaneously, pointing toward the destination. This approach establishes a foundation for future improvements:

- Scalability: multiple wheelchair agents could be added in future iterations without additional pathfinding overhead

Flow Field Algorithm — How It Works

Step 1: A uniform grid is overlaid on the hospital floor. On launch, each cell is classified as walkable or unwalkable based on obstacle detection (with a cost of 1 or 255 respectively).

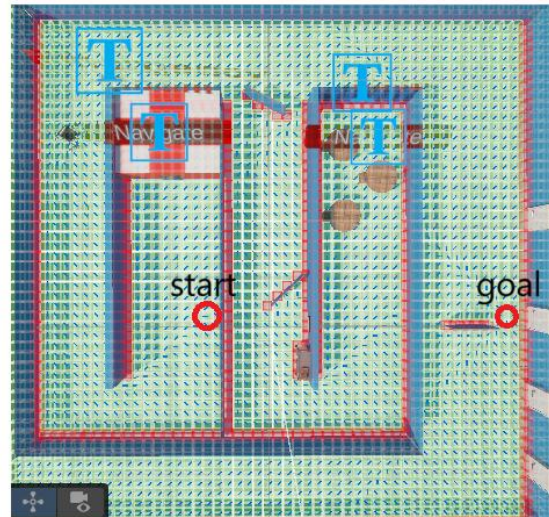
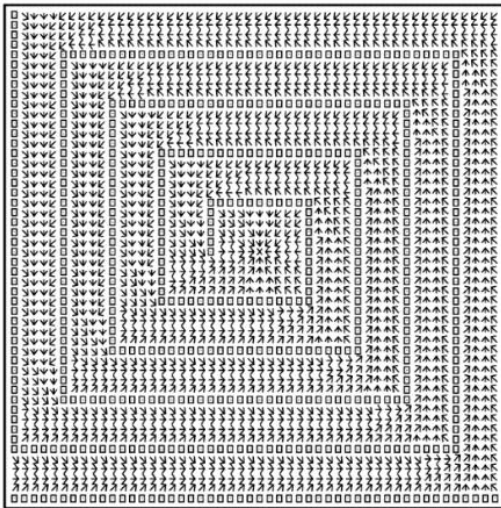
Step 2: A cost field is computed from the destination cell outward.

Step 3: Each cell calculates the best direction vector pointing toward its lowest-cost neighbor. The agent reads the vector of its current cell and moves accordingly.

Appendices:



(The grid controller implemented, allows the user to change grid's size. Decreasing cell radius and increasing total grid size results in a *more accurate navigation calculation* but *heavier computation and ram allocation*.)



(Left image: Flow Field Visualization from the Course's Lecture. Right image: Flow Field Custom Visualization during Project's runtime.)

2.3 Agent Behavior & Obstacle Avoidance

The wheelchair agent (FlowFieldAgent Object) moves by reading the flow direction of its current grid cell. Several behaviors were implemented to handle edge cases that arise in real navigation scenarios:

- Stuck detection: if the agent remains stationary on an unwalkable (red) cell for 3 consecutive seconds, a small forward force is applied along the character's facing direction to break free
- Smooth rotation: the agent uses Quaternion.Slerp to rotate toward its direction of travel, producing natural-looking movement

2.4 Hospital Environment & Interactable Locations

The 3D hospital environment was designed with three key navigable destinations, each representing a distinct patient transport scenario. World-space UI buttons are placed above each location. When clicked it triggers the Agent's movement to the respective destination using flow field pathfinding. Additionally, k obstacles (medical props) are spawned at launch, with *random rotation and world location* within the map (k locations picked randomly from a large array of possible locations).



(Sample taken from a random runtime)

2.5 Simulation System

Apart from the on-command, manual navigation buttons, at runtime, there is a button “Simulate” that triggers the below procedure: A HospitalSimulator manager script orchestrates an autonomous simulation loop. The simulation system:

- Randomly selects a destination from a *modifiable, predefined table of probabilities*
- Prevents the same location from being selected twice in a row by re-weighting the remaining possibility pool
- Displays a contextual UI panel describing the current patient transport scenario
- Moves the wheelchair agent to the selected location and waits for arrival
- Automatically repeats the cycle until the End Simulation button is pressed



(Screenshot from a running simulation)

Default Probability Table:

Location	Scenario Description
Emergency Room	Urgent transport — highest priority, 40% simulation chance
Patient Room	Standard return transport after treatment, 30% chance
Cafe	Scheduled patient meal break, 30% chance

3. Results

3.1 Navigation Performance

The Flow Field pathfinding algorithm successfully navigates the wheelchair agent across the hospital environment in all tested scenarios. The agent correctly avoids static obstacles and walls.

Key Navigation Outcomes

The agent successfully reaches all three hospital destinations across repeated simulation runs. Obstacle avoidance functions correctly for both pre-placed and procedurally spawned obstacles (walls and medical props respectively). The stuck-detection nudge mechanism resolves edge cases where the agent clips into unwalkable cells near edges or narrow corridors. Overall, among most simulation iterations, the flow field provides the optimal path for the agent to its destination. In realistic context, that is crucial in emergency scenarios.

3.2 Simulation Behavior

The autonomous simulation loop operates as intended across extended test sessions. The no-repeat constraint on location selection distributes transport requests more evenly across the three destinations, reflecting a realistic distribution of hospital transport needs.

Feature	Result
Flow field generation	Computed correctly on every destination change
Obstacle detection	100% of pre-placed and spawned obstacles detected

No-repeat location logic	Same location never selected in consecutive runs
Stuck resolution	Agent recovers from red-cell collisions within 3 seconds
Camera follow	Smooth camera tracking throughout all navigation, with zoom in/out mechanism

3.3 Limitations & Future Work

While the simulation meets its core objectives, several areas present opportunities for future development:

- Dynamic obstacle support: currently the grid is generated once at runtime; moving obstacles (such as staff or patients walking) are not accounted for in the flow field
- Multi-agent support: the flow field architecture supports multiple agents natively, but agent-to-agent collision avoidance has not been implemented
- Realistic hospital layout: the current environment is a simplified approximation; a full-scale hospital floor plan with accurate room dimensions would improve realism
- Patient boarding animation: a visual sequence showing a patient entering and exiting the wheelchair would improve the simulation narrative

3.4 Conclusion

This project successfully demonstrates how a Flow Field pathfinding algorithm can drive an autonomous wheelchair agent through a structured hospital environment in a convincing and functional simulation. The foundation laid by this project is well-suited for extension into a more complete and medically accurate simulation in future iterations.